



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

Python para Ciencia de Datos

Tema 5 **Funciones y módulos**

Máster F.P. En IA & Big Data ANALYTICS
Curso 2026-2027

UPV

Resultados de aprendizaje

- Describir el uso de funciones para estructurar el código de un programa y las ventajas que esto presenta
- Identificar los tipos de funciones en Python y saber cómo invocarlas correctamente
- Definir y usar funciones acordes con las necesidades de un programa, eligiendo los tipos de parámetros más adecuados en cada caso
- Explicar las diferencias entre los ámbitos de las variables, y utilizar correctamente variables globales y locales en un programa
- Explicar el concepto de función o expresión lambda y ser capaz de definir y usar estas funciones, en el contexto de funciones `filter()` y `map()` entre otros
- Crear módulos para distribuir el código de un programa y utilizar sus funciones correctamente desde otros módulos

Índice



- 1. Funciones**
- 2. Ámbitos de las variables**
- 3. Funciones lambda**
- 4. Módulos**

Índice



1. Funciones

2. Ámbitos de las variables

3. Funciones lambda

4. Módulos

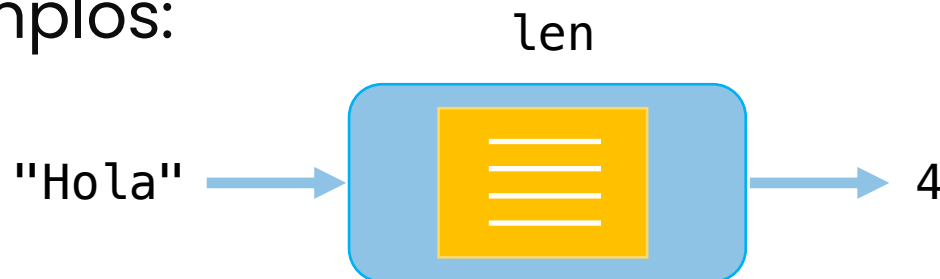
Funciones

Concepto

- Una **función** es una secuencia de **instrucciones**, agrupadas bajo un cierto **nombre**, que realiza una **tarea** específica
- Normalmente, acepta ciertos datos como **entrada** y produce otros datos como **salida**



- Ejemplos:



Funciones

Ventajas

- Las funciones se ejecutan (solo) cuando las **invocamos**, y tantas veces como necesitemos
- Facilitan la **reutilización del código**, aumentando la productividad del programador
- Permiten abordar problemas complejos, **dividiéndolos** en subproblemas más simples
- Mejoran la **estructura** del programa, haciéndolo más legible y comprensible
- Facilitan el **mantenimiento** del programa

Funciones

Tipos de funciones

- Python (como otros lenguajes) admiten **3 tipos de funciones**:

- 1. Funciones integradas (*built-in functions*)**

- Forman parte del propio lenguaje (Python)
- Se pueden usar directamente, sin importar ningún módulo
- Ejemplos: `print`, `input`, `abs`, `min`, `max`, `len`, ...

- 2. Funciones de biblioteca (*library functions*)**

- Incluidas dentro de módulos (externos) específicos
- Para usarlas es necesario importar primero su correspondiente módulo
- Ejemplos: `math.sqrt`, `random.randint`, ...

- 3. Funciones definidas por el usuario**

- Creada por los programadores

En este tema
aprenderemos a
crearlas y usarlas

Funciones

Invocación de funciones integradas

- Vamos a presentar cómo se **invocan** (o llaman) funciones desde nuestro código, comenzando por las funciones **integradas**:
 - Para llamar o invocar una función escribimos su **nombre** seguido de una lista de **argumentos** entre paréntesis
 - Cada argumento es un **valor** (o una **expresión**) del **tipo de dato** concreto que espera la función
 - Los **paréntesis** son necesarios incluso cuando la función no necesita ningún argumento

```
abs(-10)
len("Hola")
min(7, 3, 5)
```

```
abs(x**2 - 5*x + 1)
len("Hola" + " mundo!")
min(x, y)
```

```
exit()
```


Funciones

Invocación de funciones integradas

- Vamos a presentar cómo se **invocan** (o llaman) funciones desde nuestro código, comenzando por las funciones **integradas**:
 - De igual forma, las funciones que **devuelven un valor** pueden ser usadas como parte de una expresión

```
z = y + abs(3*x**2 - 5*x + 1) / 2  
print(f'Solución: {(-b + math.sqrt(b**2 - 4*a*c)) / (2*a)}')
```

- La **documentación** que detalla cómo se debe invocar una función (su nombre, argumentos, operación que realiza, y valores de retorno) se conoce como el **Application Program Interface** (o **API**): <https://docs.python.org/3/library/functions.html>

Funciones

Invocación de funciones de biblioteca

- Respecto a las funciones **de biblioteca**:
 - Son funciones que, aunque no forman parte del propio lenguaje, están disponibles y organizadas en **módulos** (por ejemplo, `math`, `random`, `numpy`, `pandas`, etc.)
 - Por ejemplo, el módulo `math` contiene funciones que realizan las operaciones matemáticas más habituales (`sqrt()`, `sin()`, etc.)
 - En el caso de Python, el número y variedad de estas bibliotecas es enorme, aumentando considerablemente su potencia
 - A diferencia de las funciones integradas, para poder utilizar estas funciones es necesario **importarlas** desde su módulo

Funciones

Invocación de funciones de biblioteca

- Hay tres formas de importar módulos y funciones:

1. Importar un **módulo completo** (con todas sus funciones):

```
import math      # Importa el módulo math
x = math.sqrt(2) # Llama a la función sqrt() de ese módulo
```

2. Importar todo un módulo y **renombrarlo** con un **alias**:

```
import math as mt # Importa el módulo math con el alias mt
x = mt.sqrt(2)    # Llama a sqrt() de ese módulo usando el alias
```

3. Importar una **función específica** contenida en un módulo:

```
from math import sqrt # Importa sqrt() desde el módulo math
x = sqrt(2)           # Llama a la función importada sqrt()
```

Funciones

- **Ejemplo:** Crea dos puntos aleatorios y obtiene la distancia mínima desde el origen

```
import random as rnd
import math

# Posiciones del punto 1 (x1, y1) y del punto 2 (x2, y2)
x1 = rnd.randint(0, 50) # Llamada a la función randint() dentro del módulo rnd (random)
y1 = rnd.randint(0, 50) # para obtener un valor entero aleatorio entre 0 y 50
x2 = rnd.randint(0, 50)
y2 = rnd.randint(0, 50)

# Cálculo de la distancia al origen
d1 = math.sqrt(x1**2 + y1**2) # Llamada a la función sqrt() ubicada en el módulo math
d2 = math.sqrt(x2**2 + y2**2)

# Cálculo de la distancia mínima al origen
d_min = min(d1, d2) # Llamada a la función integrada min()
print(f'Mínima distancia: {d_min}')
```

Funciones

Funciones definidas por el usuario

- La **sintaxis** para crear una función definida por el usuario es:

```
def nombre_funcion (parámetros):
```

```
    instrucción1  
    instrucción2  
    ...  
    instrucciónN
```

Cabecera de la función: contiene el **nombre** de la función y sus **parámetros** entre paréntesis y separados por comas

Cuerpo de la función: secuencia de **instrucciones** que se ejecutan cuando se invoca a la función

Funciones

Funciones definidas por el usuario

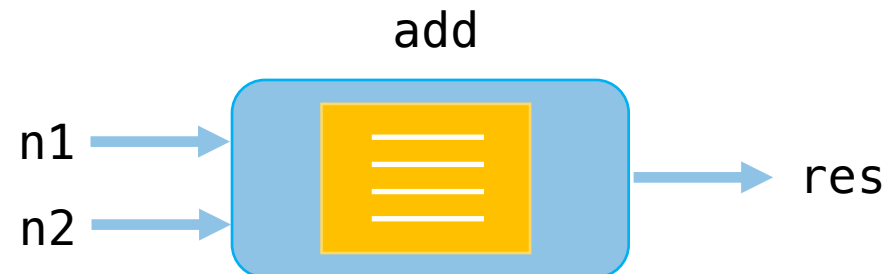
- En la **cabecera**:
 - El **nombre** es un **identificador**, y no puede coincidir con ninguna palabra reservada de Python
 - Los **parámetros** son una lista de **cero o más nombres de variable** que almacenarán los datos de entrada cuando se invoque a la función
- Hay que diferenciar entre:
 - **Parámetro**: variable de entrada definida en la **cabecera** de una función
 - **Argumento**: valor correspondiente a un parámetro en la **llamada** a la función

Funciones

Funciones definidas por el usuario

- En el **cuerpo**:
 - Las instrucciones se escriben **indentadas** bajo la cabecera
 - Igual que en las sentencias de control, la **indentación** es lo que determina el bloque de instrucciones que pertenecen a la función
 - La ejecución de la función **finaliza**:
 - Tras la última instrucción del bloque indentado, o bien
 - Mediante la instrucción **return**, que permite además **devolver** un resultado a la llamada

```
def add (n1, n2):  
    res = n1 + n2  
    return res
```



Funciones

Transferencia de información y flujo de control

- Cuando **invocamos** una función, se evalúa cada **argumento** indicado en la llamada y su valor se asigna al **parámetro** (variable de entrada) correspondiente de la cabecera de la función
- Si la función ejecuta una instrucción **return**, el valor indicado se devuelve, reemplazando la propia llamada

Funciones

Transferencia de información y flujo de control

- Cuando **invocamos** una función, se evalúa cada **argumento** indicado en la llamada y su valor se asigna al **parámetro** (variable de entrada) correspondiente de la cabecera de la función

```
def funcion(param1, ..., paramN):  
    instrucción1  
    instrucción2  
    ...  
    return resultado
```

```
func_name(arg1, ..., argN)
```



Funciones

Transferencia de información y flujo de control

- Si la función ejecuta una instrucción **return**, el valor indicado se devuelve, reemplazando a la propia llamada
- Es decir, si la llamada se encuentra formando parte de una expresión, su evaluación produce el valor devuelto por la función

```
def funcion(param1, ..., paramN):  
    instrucción1  
    instrucción2  
    ...  
    return resultado
```

```
func_name(arg1, ..., argN)
```

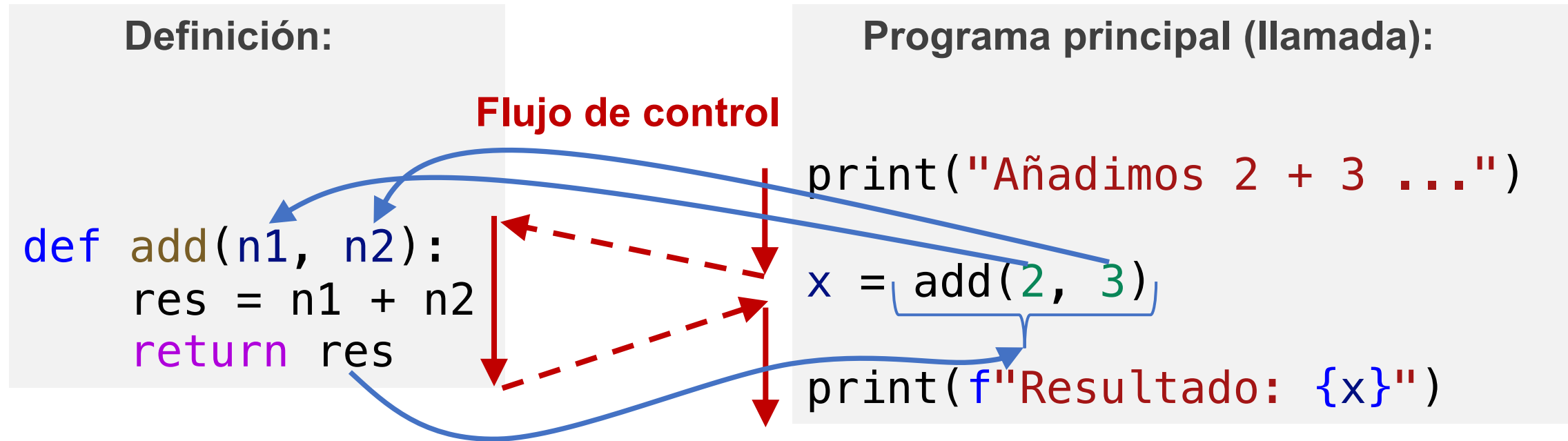


A blue curved arrow originates from the `return resultado` line in the function definition and points to the function call `func_name(arg1, ..., argN)`, illustrating the transfer of the return value.

Funciones

Transferencia de información y flujo de control

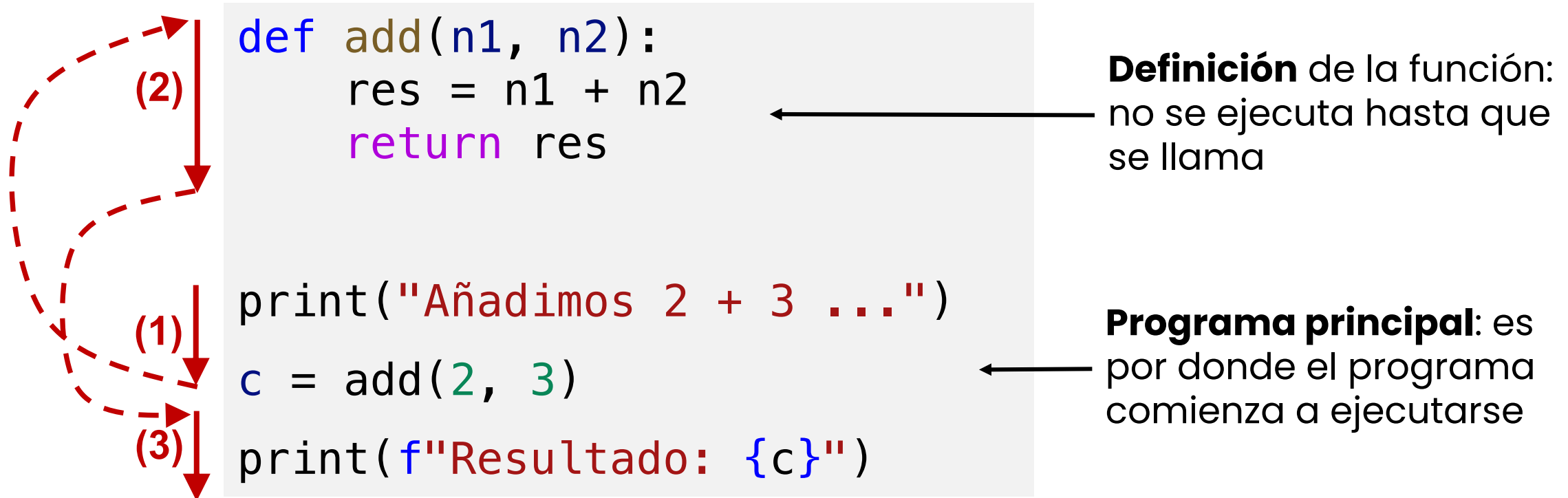
- La **invocación** produce un **salto** del flujo de control al inicio de la función, y al finalizar su ejecución, el programa **retorna** al punto de la llamada



Funciones

Transferencia de información y flujo de control

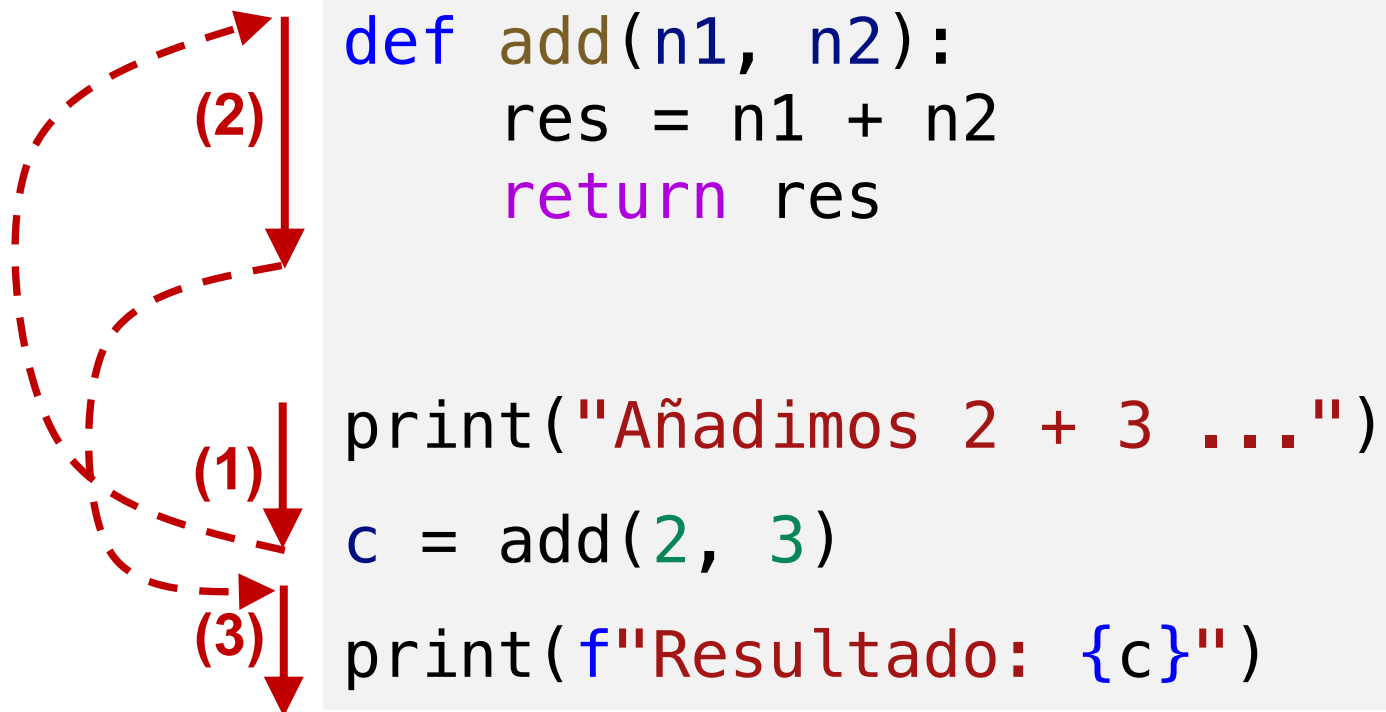
- Las funciones se definen con **anterioridad** al código o **programa principal** en donde se usan (invocan):



Funciones

Transferencia de información y flujo de control

- Las funciones se definen con **anterioridad** al código o **programa principal** en donde se usan (invocan):



Añadimos 2 + 3 ...
Resultado: 5

Funciones

Transferencia de información y flujo de control

- Ejemplos:

```
def add(n1, n2):  
    res = n1 + n2  
    return res
```

Los **parámetros** son variables dentro del código de la función

```
x = add(2**3, math.sqrt(5))  
print(f'Result: {x}')
```

```
x = add(5, 10) / 2  
print(f'Result: {x}')
```

```
print(f'Result: {add(2,3)}')
```

```
x = add(add(2,3), 4)  
print(f'Result: {x}')
```

```
if add(2,3) == 5:  
    print('Result: 5')
```

```
a = float(input('>> '))  
b = float(input('>> '))  
x = add(a, b)  
print(f'Result: {x}')
```

```
Result: 10.23606797749979  
Result: 7.5  
Result: 5  
Result: 9  
Result: 5  
>> 2.3  
>> 4.5  
Result: 6.8
```

Los **argumentos** son expresiones que se evalúan antes de la llamada

Funciones

Funciones sin parámetros

- Algunas funciones no requieren ningún parámetro
- Incluso en ese caso, indicaremos los paréntesis en la llamada
- Ejemplo:

```
import random

def consecutive_heads_flipping_coin():
    count = 0
    while random.choice(['Head', 'Tail']) == 'Head':
        count += 1
    return count

ch = consecutive_heads_flipping_coin()
print(f'You have obtained {ch} consecutive heads')
```

Funciones

Funciones sin retorno

- El objetivo de algunas funciones no es devolver el resultado de un cálculo, sino realizar alguna acción observable en el sistema
- En esos casos, la función **no requiere** la instrucción `return` y termina al finalizar el bloque de instrucciones
- Si en el programa principal la llamada está formando parte de una expresión, se evalúa al valor **None**

- Ejemplo:

```
def multiplication_table(n):  
    for x in range(1,11):  
        print(f'{n} x {x} = {n*x}')
```

`multiplication_table(7)`

```
7 x 1 = 7  
7 x 2 = 14  
7 x 3 = 21  
7 x 4 = 28  
...
```


Funciones

Funciones con más de un retorno

- En una instrucción puede existir **más de una** instrucción **return** y en ese caso la función finaliza en cuanto se ejecuta la primera de ellas
- **Ejemplo:** compara estas dos funciones (son equivalentes):

```
def absolute_value(x):  
    if x < 0:  
        return -x  
    else:  
        return x
```

```
def absolute_value(x):  
    if x < 0:  
        return -x  
    return x
```

↑
El else no es necesario

Funciones

Valores múltiples en un mismo retorno

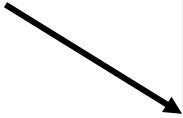
- Una misma instrucción **return** puede devolver **múltiples valores** separados por comas:

```
return expr1, expr2, ... , exprN
```

- Ejemplo:

```
def solve_quadratic_equation(a, b, c):  
    disc = b**2 - 4*a*c  
    if (a == 0 and b == 0) or disc < 0:  
        return None, None  
    if a == 0 and b != 0:  
        return -c/b, None  
    r = math.sqrt(disc)  
    return (-b + r) / (2*a), (-b - r) / (2*a)  
  
sol1, sol2 = solve_quadratic_equation(1,4,2)
```

Los valores se devuelven como una **tupla** y se pueden asignar a varias variables directamente



Funciones

Tipos de parámetros

- En principio, la **invocación** de una función debe incorporar **tantos** argumentos como parámetros tiene en su definición, y en el **mismo orden** en que están definidos esos parámetros
- Python permite definir e invocar funciones con distintos tipos de parámetros/argumentos:
 - Parámetros **opcionales** (con valor por defecto)
 - Argumentos **posicionales** o **con nombre**
 - Parámetros **arbitrarios posicionales**
 - Parámetros **arbitrarios con nombre**

Funciones

Parámetros opcionales (valor por defecto)

- Es posible indicar que un parámetro es **opcional** asignándole un **valor predeterminado** en la definición de la función
- Si una función tiene parámetros con valor por defecto, deben ubicarse siempre **al final de la lista**, tras los que no tienen valor
- Ejemplo:

```
def saluda(nombre, saludo = "Hola"):
    print(f'¡{saludo}, {nombre}!')
```

```
saluda("Alicia")
saluda("Pepe", "Buenos días")
```

```
¡Hola, Alicia!
¡Buenos días, Pepe!
```

Funciones

Argumentos con nombre

- En la **llamada** a una función, es posible indicar el **nombre** del parámetro junto con su valor (**argumento con nombre**)
- Eso permite que el **orden** de los argumentos **no coincida** con el orden de los parámetros
- Ejemplo:

```
def saluda(nombre, saludo = "Hola"):  
    print(f'¡{saludo}, {nombre}!')  
  
saluda("Alicia")  
saluda(saludo = "Buenos días", nombre= "Pepe")
```

```
¡Hola, Alicia!  
¡Buenos días, Pepe!
```

Funciones

Argumentos con nombre vs posicionales

- En una misma llamada es posible incluir argumentos sin nombre (se identifican por su posición) y otros con nombre
- Si en una misma llamada hay argumentos **posicionales** y otros con nombre, los que llevan **nombre** se ubican siempre **al final**
- Los argumentos con nombre son útiles en situaciones en las que:
 - Una función tiene muchos parámetros con valor por defecto
 - En la llamada queremos cambiar solo alguno de esos valores

Funciones

Argumentos con nombre vs posicionales

- **Ejemplo:** calcular el precio de un producto a partir de su precio base, un posible descuento y un cierto valor de IVA

```
def calcular_precio(base, iva = 0.21, descuento = 0.0):  
    return (base - base * descuento) * (1.0 + iva)
```

```
print(calcular_precio(100))  
print(calcular_precio(100, 0.05, 0.5))  
print(calcular_precio(100, descuento = 0.5))
```

```
121.0  
52.5  
60.5
```

El parámetro base es obligatorio y debe ir al principio, y descuento e iva son opcionales (y van al final). En la última llamada, se especifica descuento por nombre para evitar tener que dar valor también a iva

Funciones

Parámetros arbitrarios posicionales

- Python también permite definir **parámetros arbitrarios**, que pueden recibir **distinto número** de argumentos en cada llamada

■ Ejemplo:

```
def media(*numeros):  
    n = total = 0  
    for i in numeros:  
        total += i  
        n += 1  
    return total / n  
  
print(media(10))  
print(media(10, 20))  
print(media(10, 20, 30, 40))
```

```
10.0  
15.0  
25.0
```

El parámetro se especifica con un * (en este caso `*numeros`) y funciona como una **tupla**

Funciones

Parámetros arbitrarios por nombre

- Finalmente, los **parámetros arbitrarios** también pueden pasarse por **nombre**, lo que internamente se trata como un **diccionario**

```
def nombre_completo(**n):  
    if 'ap2' in n:  
        return n['ap1'] + ' ' + n['ap2'] + ', ' + n['pila']  
    else:  
        return n['ap1'] + ', ' + n['pila']  
  
nombre = {'pila': 'Antonia', 'ap1': 'Pérez', 'ap2': 'García'}  
  
print(nombre_completo(pila='John', ap1='Snow'))  
print(nombre_completo(**nombre))
```

Snow, John
Pérez García, Antonia

Los argumentos se pasan **por nombre** como si fueran las **claves** de un **diccionario**

Funciones

Parámetros: uso de / y *

- En el listado de parámetros, podemos usar los caracteres / y * con un significado especial:

```
def f(pos1, pos2, /, pos_or_kwd, *, kwd1, kwd2):
```

- / **separa** los parámetros que deben recibir argumentos posicionales necesariamente, de otros que pueden recibir argumentos posicionales o por nombre
- * **separa** los parámetros que pueden recibir argumentos posicionales o por nombre, de otros que solo pueden recibirlos por nombre
- Ejemplo: `max(iterable, /, *, key=None)`

Funciones

Parámetros: recomendaciones

- ¿Cuándo es útil **forzar** el uso de parámetros de un tipo u otro?
 - Usar parámetros **exclusivamente posicionales** si no queremos que sus nombres estén disponibles en la llamada, o si queremos forzar su uso en un cierto orden
 - Usar parámetros **exclusivamente por nombre** cuando sus nombres tengan un significado definido, o bien cuando no se quiera confiar en que se usarán en el orden definido en la llamada

Índice



1. Funciones

2. Ámbitos de las variables

3. Funciones lambda

4. Módulos

Ámbitos de las variables

Alcances posibles de una variable

- El **ámbito** o **alcance** de una variable es la parte del código del programa en la que existe y puede accederse a ella
- Tenemos básicamente **dos ámbitos** o alcances:
 1. **Ámbito local**: las variables definidas **dentro** de una función tienen como alcance el código de dicha función y no fuera de ella. Se dice que son **variables locales** (de dicha función)
 2. **Ámbito global**: las variables definidas **fuera** de las funciones (en el programa principal) tienen como alcance el código completo del script o módulo en el que están definidas. Se les suele denominar **variables globales**

Ámbitos de las variables

Ámbito local

- Las variables definidas dentro de una función no están disponibles fuera de esa función. Cualquier acceso a ellas provoca un **error**:

```
1 def add(n1, n2):  
2     res = n1 + n2  
3     return res  
4  
5 add(2, 3)  
6 print(f'Result: {res}') # Error: ires tiene ámbito local!
```

```
----> 6 print(f'Result: {res}')
```

```
NameError: name 'res' is not defined
```

Ámbitos de las variables

Ámbito global

- Las variables definidas en un **ámbito global** sí pueden accederse desde el código de una función, pero esto se **desaconseja** porque produce un acoplamiento (dependencia) indeseable entre el código de la función y el programa principal

```
1 def add():
2     res = a + b # ¡TOTALMENTE desaconsejado!
3     return res
4
5 # a y b tienen ámbito global
6 a = float(input("Primer número:"))
7 b = float(input("Segundo número:"))
8 result = add()
9 print(f'Result: {result}')
```

La **comunicación** con una función debe realizarse solo mediante sus **parámetros de entrada** y los valores de **retorno**

Ámbitos de las variables

Misma variable en distintos ámbitos

- Si dentro de una función, una variable definida globalmente recibe un valor, se **crea** una **variable local** con el **mismo nombre**, y la variable global no se modifica

```
1 def add_number():  
2     i = 100  
3     return i  
4  
5 i = 50  
6 print(add_number())  
7 print(i)
```

```
100  
50
```

Cambiamos el valor de la variable `i`, pero no es la misma que la global (la variable local “tapa” u oculta la global)

Tras la ejecución de la función, `i` sigue valiendo `50`

Ámbitos de las variables

Misma variable en distintos ámbitos

- Pero si dentro de la función definimos la variable como **global**, entonces sí se accede a la misma variable del programa principal

```
1 def add_number():  
2     global i  
3     i = 100  
4     return i
```

```
5  
6 i = 50  
7 print(add_number())  
8 print(i)
```

global es una palabra reservada que define una variable (de una función) como global

Tras la ejecución de la función, i vale 100

```
100  
100
```

Ámbitos de las variables

Misma variable en distintos ámbitos

- **OJO:** cuando la variable almacena un tipo estructurado (mutable), como una lista, entonces la variable local (copia) sí accede a la **misma estructura**, y se modifica también la variable global

```
1 def add_list():  
2     lista.append(100)  
3  
4 lista = [50]  
5 add_list()  
6 print(lista)
```

lista es una copia de la **referencia** a la estructura, con lo que se accede a la misma estructura en memoria

Tras la ejecución de la función, lista se ha modificado

```
[50, 100]
```

Ámbitos de las variables

Resumen

- Desde un **código global** no puede accederse a ninguna variable definida dentro de **ámbito local** (función)
- Por el mismo motivo, las funciones no pueden acceder a ninguna variable definida dentro de **otra función**
- Las funciones sí pueden acceder a **variables globales**, pero se **desaconseja** encarecidamente
- Si una función define una **variable local** con el mismo nombre que una global, esta queda “oculta” por la variable local, a no ser que esté definida mediante la palabra reservada **global**
- Este último caso también se desaconseja

Índice



1. **Funciones**
2. **Ámbitos de las variables**
3. **Funciones lambda**
4. **Módulos**

Funciones lambda

Concepto

- Una **función lambda** (o una ***expresión lambda***) es una función **anónima** que contiene una **única expresión**
- Suele ser útil para situaciones en las que requerimos de una función **simple** y de **un solo uso**
- Definición:

lambda [p1, p2, ...] : expresión

donde:

- p1, p2, ... son los **parámetros de entrada**, y la función devuelve por defecto el resultado de **evaluar** la expresión

Funciones lambda

Invocación

- Una expresión lambda puede especificarse en el **mismo momento** en que se invoque, indicando la expresión entre paréntesis, seguida por los argumentos (también entre paréntesis)

- Por ejemplo:

```
z = (lambda x, y: x + y)(5, 10)
```

```
print(z)
```

 15

- Así evitamos tener que definir una función:

```
def suma(x, y):  
    return x + y
```

 15

```
z = suma(5, 10)  
print(z)
```

Funciones lambda

Utilidad

- Las expresiones lambda son útiles en situaciones en las que se requiere definir función que haga algo simple y no reutilizable
- Un ejemplo son las funciones predefinidas `filter()` y `map()`, que requieren entre sus parámetros una **referencia a una función** como parte de su definición
- A continuación, se presentan brevemente ambas funciones

Funciones lambda

`filter()`

- La función `filter()` aplica un **filtro** a los elementos de un **objeto iterable (secuencia)**, y devuelve otro iterable que contiene solo los elementos que “pasan el filtro”
- El filtro se define como una **función** que, aplicada a un elemento (de la secuencia), devuelve **True** o **False**
- Sintaxis:

```
filter(función_filtro, iterable)
```

donde:

- La `función_filtro` puede ser una función “normal” o lambda
- El `iterable` puede ser cualquier secuencia (lista, tupla, cadena,...)

Funciones lambda

filter()

- Ejemplo con una función normal:

```
def es_par(x):  
    return True if x % 2 == 0 else False
```

```
[2, 4, 6]
```

```
lista = [1, 2, 3, 4, 5, 6]  
pares = filter(es_par, lista)  
print(list(pares))
```

filter() devuelve un "iterable",
que luego podemos convertir,
por ejemplo, a una lista

- Mismo ejemplo, con lambda:

```
lista = [1, 2, 3, 4, 5, 6]  
pares = filter(lambda x: x % 2 == 0, lista)  
print(list(pares))
```

```
[2, 4, 6]
```

Funciones lambda

map ()

- La función map () aplica un **cálculo** a los elementos de un o varios **iterables**, y devuelve otro iterable con el resultado de aplicar dicho cálculo a cada elemento
- El cálculo se define como una **función** que, aplicada a un elemento de **cada** iterable, devuelve un valor (de cualquier tipo)
- Sintaxis:

```
map(función, iterable1, ...)
```

donde:

- La función puede ser una función "normal" o lambda, y acepta tantos argumentos como iterables se especifiquen después

Funciones lambda

map()

- Ejemplos:

```
lista = [1, 2, 3, 4, 5, 6]
cuadrados = map(lambda x: x ** 2, lista)
print(list(cuadrados))
```

```
[1, 4, 9, 16, 25, 36]
```

```
lista1 = [1, 2, 3, 4, 5, 6]
lista2 = [10, 20, 30, 40, 50]

sumas = map(lambda x, y: x + y, lista1, lista2)
print(list(sumas))
```

```
[11, 22, 33, 44, 55]
```

La ejecución de map() finaliza en cuanto se acaba cualquiera de las secuencias

La expresión lambda tiene 2 parámetros, uno por secuencia

Índice



- 1. Funciones**
- 2. Ámbitos de las variables**
- 3. Funciones lambda**
- 4. Módulos**

Módulos

Definición

- Un programa corto suele escribirse en un único fichero .py, pero cuando aumenta de complejidad y tamaño, suele ser conveniente **dividirlo** entre **distintos ficheros** .py
- Un **módulo** es un archivo que contiene código Python y que puede ser usado desde otro módulo (o programa)

modulo1.py

```
def f1():  
    ...  
  
def f2():  
    ...
```

modulo2.py

```
def f1():  
    ...  
  
def f2():  
    ...
```

mi_programa.py

```
from modulo1 import f1  
from modulo2 import f3  
  
x = f1()  
y = f3()
```

El **nombre de un módulo** es el mismo nombre del archivo en el que se define, **sin** la extensión .py.

Módulos

Ventajas

- Los módulos ayudan a **organizar el código** agrupando las funciones relacionadas
- Las **funciones** y **variables** definidas en un módulo se pueden importar de otros módulos para su uso
- El uso de módulos facilita la **reutilización** de código en diferentes proyectos
- Los módulos proporcionan un **espacio de nombres independiente** para variables y funciones, lo que ayuda a evitar conflictos de nomenclatura

Módulos

Invocación de funciones

- Como ya hemos visto, invocar una función definida en el **mismo** módulo o programa (fichero .py) es **directo**
- Si la función que queremos invocar se define en **otro** módulo (escrito por nosotros o de una biblioteca de terceros), tenemos que **importar** la función (o el módulo completo)

modulo1.py

```
def f1():  
    ...  
  
def f2():  
    ...
```

modulo2.py

```
def f1():  
    ...  
  
def f2():  
    ...
```

mi_programa.py

```
import math  
import modulo1 as m1  
from modulo2 import f3  
  
x = m1.f1()  
y = f3()  
z = math.sqrt(100)
```

Observa las tres posibles formas de importar un módulo/función

Módulos

Código global en un módulo

- El **código global** de un módulo se **ejecuta** cuando ejecutamos el módulo, pero también cuando ese módulo se **importa** desde otro

temperature_conversion.py

```
def cel_to_fahr(c):  
    return c * 9/5 + 32
```

```
def fahr_to_cel(f):  
    return (f-32) * 5/9
```

```
celsius = float(input('Enter °C degrees:'))  
fahrenheit = cel_to_fahr(celsius)  
print(f'{celsius}°C = {fahrenheit}°F')
```

¡Cuidado! Este código se ejecutará cuando **importemos** el módulo

Módulos

Código global en un módulo

- Para evitarlo, podemos incluir el código global en una **función**:

temperature_conversion.py

```
def cel_to_fahr(c):  
    return c * 9/5 + 32  
  
def fahr_to_cel(f):  
    return (f-32) * 5/9  
  
def main():  
    celsius = float(input('Enter °C degrees:'))  
    fahrenheit = cel_to_fahr(celsius)  
    print(f'{celsius}°C = {fahrenheit}°F')
```

Ahora el código ya **no** es **global**: no se ejecutará al importar el módulo, pero **tampoco** cuando se ejecute directamente su fichero .py

Módulos

Código global en un módulo

temperature_conversion.py

```
def cel_to_fahr(c):  
    return c * 9/5 + 32  
  
def fahr_to_cel(f):  
    return (f-32) * 5/9  
  
def main():  
    celsius = float(input('Enter °C degrees:'))  
    fahrenheit = cel_to_fahr(celsius)  
    print(f'{celsius}°C = {fahrenheit}°F')  
  
if __name__ == '__main__': main()
```

Este **código global** consigue que un cierto código (el de la función `main()`) se ejecute si el fichero `.py` se ejecuta como un **programa**, pero **no** si se importa como un **módulo**

Módulos

Ubicación del módulo

- Para que un módulo sea importado, debe estar ubicado en un directorio **accesible** para Python
- Búsqueda de módulos en Python se realiza en:
 - El directorio donde se encuentra el script que importa el módulo
 - Los directorios especificados por la variable de entorno PYTHONPATH
 - Los directorios predeterminados del sistema donde están instaladas las bibliotecas de Python